Playwright Interview Questions with Detailed Answers (Advanced 2026 Edition)

---

1. What makes Playwright different from Selenium?

Playwright is a modern end-to-end automation framework developed by Microsoft.
Key differences:
• Built-in auto-waiting mechanism (reduces flaky tests).
• Supports Chromium, Firefox, and WebKit with a single API.
• No need for separate browser drivers.
• Native parallel execution.
• Better handling of modern web apps (SPA, Shadow DOM).
• Built-in tracing, video recording, and screenshots.
In contrast, Selenium requires WebDriver setup and explicit waits more frequently.

2. Explain Auto-Waiting in Playwright.

Playwright automatically waits for elements to be:
• Attached to the DOM
• Visible
• Stable (not animating)
• Enabled and actionable
Before performing actions like click(), fill(), or hover().
This eliminates the need for Thread.sleep() or explicit waits in most cases, making tests stable and reliable.

3. Difference between page.locator() and page.$()?

page.locator():
• Recommended approach.
• Supports auto-waiting and retry logic.
• Lazy evaluation (evaluated when action is performed).
• Works well with assertions.

page.$():
• Returns element handle immediately.
• No retry or auto-wait.
• Can lead to flaky tests if element is not ready.

4. How do you handle Iframes in Playwright?

Playwright provides frameLocator() for interacting with iframes.
Example:
const frame = page.frameLocator('#frameId');
await frame.locator('button').click();

This ensures reliable interaction inside embedded frames.

5. How do you validate API responses in UI automation?

Use page.waitForResponse() to capture network calls.
Example:
const response = await page.waitForResponse('**/api/login');
expect(response.status()).toBe(200);

You can validate:
• Status codes
• Response body
• Headers
• Response time
This helps combine UI + API validation in a single test.

6. Explain Parallel Execution in Playwright.

Playwright Test Runner runs tests in parallel by default using worker processes.
Configuration:
workers: 4
Fully isolated browser contexts ensure no shared session or cookie conflicts.

7. How do you implement Page Object Model (POM)?

Create separate classes for each page.
Example:
class LoginPage {
constructor(page){
this.page = page;
this.username = page.locator('#username');
}
async login(user, pass){
await this.username.fill(user);
}
}
This improves maintainability and reusability.

8. How to capture Screenshots & Videos?

In playwright.config.js:
use: {
screenshot: 'only-on-failure',
video: 'on-first-retry',
trace: 'on-first-retry'
}
This helps debugging failed executions.

9. How to integrate Playwright in CI/CD?

Steps:
1. Install dependencies (npm install).
2. Add command: npx playwright test.
3. Store reports as artifacts.
4. Run in headless mode.
Supported platforms: Jenkins, GitHub Actions, GitLab CI.

10. How to handle Multi-User Scenarios?

Use multiple browser contexts:
const user1 = await browser.newContext();
const user2 = await browser.newContext();

Each context simulates a separate user session with isolated cookies and storage.

11. How to test File Upload and Download?

Upload:
await page.setInputFiles('#upload', 'file.pdf');

Download:
const download = await page.waitForEvent('download');
await download.saveAs('path/file.pdf');

12. How do you reduce flaky tests?

Best Practices:
• Use locators instead of element handles.
• Avoid hard waits.
• Use assertions with auto-wait.
• Use retries in configuration.

• Stabilize test data and environment.

13. How do you generate Reports?

Use built-in HTML reporter:
reporter: [['html', { open: 'never' }]]
Generates detailed execution report with steps and traces.

14. How do you test Shadow DOM?

Playwright automatically pierces Shadow DOM using locators.
Example:
await page.locator('custom-element >> text=Submit').click();

15. Real-world Example Question: How will you automate Login with OTP?

Strategy:
• Enter credentials.
• Wait for OTP API response.
• Fetch OTP from test environment (mock/email/db).
• Validate successful login.
• Add negative tests for invalid/expired OTP.